

CSS Flexbox

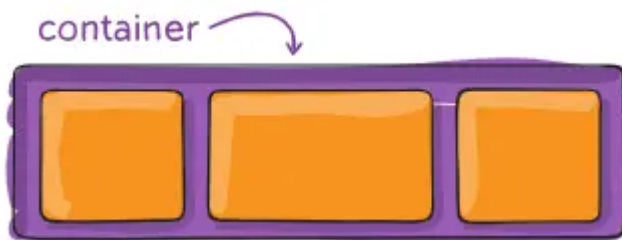
Introducción

CSS Flexbox (Flexible Box Layout) es un sistema de diseño unidimensional que nos permite distribuir el espacio entre los elementos de una interfaz y mejorar las capacidades de alineación. Es especialmente útil cuando no conocemos el tamaño de nuestros elementos o cuando este es dinámico.

Conceptos Básicos

Flexbox está formado por dos tipos de elementos:

- **Contenedor padre (flex container):** El elemento que contiene los elementos flexibles



- **Elementos hijo (flex items):** Los elementos que están dentro del contenedor flexible



Estructura HTML Típica

```
<div class="contenedor">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>
```



```
.contenedor {
  display: flex;
}
```



Características Principales

- **Sistema unidimensional:** Trabaja en una sola dimensión a la vez (horizontal o vertical)
- **Distribución del espacio:** Permite distribuir eficientemente el espacio disponible
- **Alineación avanzada:** Ofrece múltiples opciones de alineación
- **Flexibilidad:** Los elementos pueden crecer, encogerse o mantener su tamaño

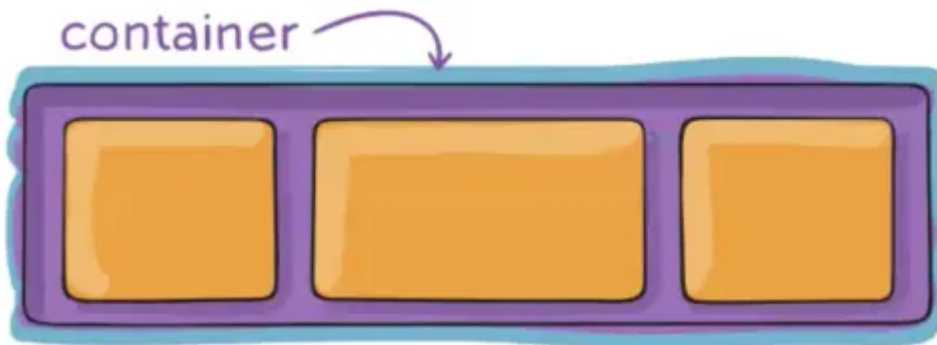
Display

La propiedad `display` es fundamental para activar Flexbox en un contenedor:

`display: flex`

Convierte el elemento en un contenedor flexible de tipo bloque (y por tanto, ocupa todo el ancho disponible).

```
.contenedor {  
  display: flex;  
}
```



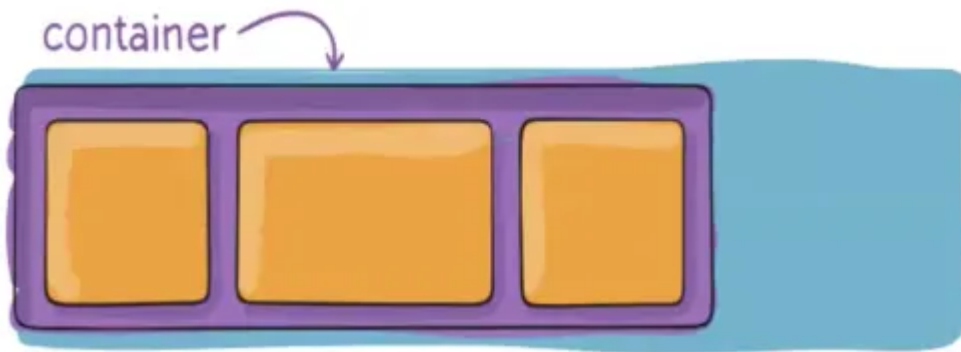
Características:

- El contenedor ocupa todo el ancho disponible
- Se comporta como un elemento de bloque
- Los elementos hijos se convierten automáticamente en flex items

`display: inline-flex`

Convierte el elemento en un contenedor flexible de tipo inline:

```
.contenedor {  
  display: inline-flex;  
}
```



Características:

- El contenedor solo ocupa el espacio necesario para sus hijos
- Se comporta como un elemento inline
- Permite que otros elementos se coloquen a su lado
- Los elementos hijos también se convierten en flex items

Diferencias Visuales

```
<div class="flex-block">Contenedor flex (bloque)</div>  
<div class="flex-inline">Contenedor inline-flex</div>  
<span>Texto después</span>
```

```
.flex-block {  
  display: flex;  
  background-color: lightblue;  
}  
  
.flex-inline {  
  display: inline-flex;  
  background-color: lightgreen;  
}
```

Wrap

Cuando creamos un contenedor y le añadimos ítems, nos podemos encontrar que no tengamos espacio suficiente para todos. Para estas situaciones, tenemos que decidir qué comportamiento seguir.

Con flexbox, podemos utilizar la siguiente propiedad:

```
flex-wrap: wrap;
```



Valores de flex-wrap

La propiedad acepta los siguientes valores:

nowrap (valor por defecto)

```
.contenedor {  
  display: flex;  
  flex-wrap: nowrap;  
}
```



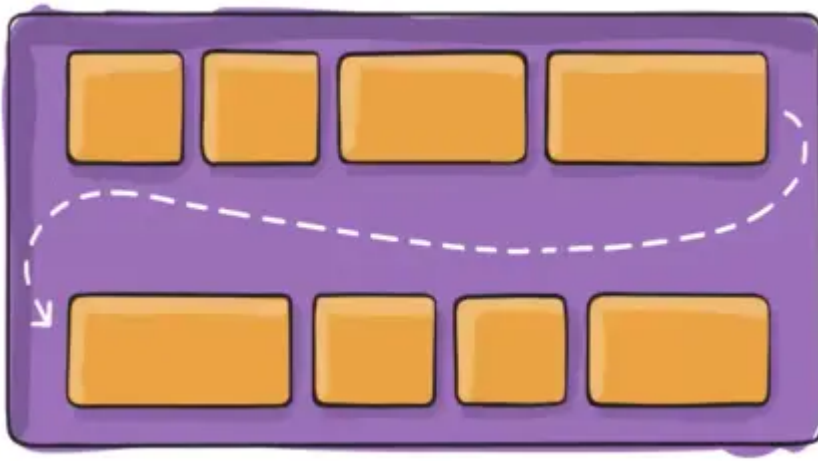
- Indica al navegador que no cree nuevas filas
- El contenido se puede salir del contenedor
- Los elementos se comprimen para caber en una sola línea

wrap

```
.contenedor {  
  display: flex;  
  flex-wrap: wrap;  
}
```



- Si no entran los elementos, el contenedor va creando nuevas filas
- Los elementos mantienen su tamaño natural
- Las nuevas filas se crean hacia abajo



wrap-reverse

```
.contenedor {  
  display: flex;  
  flex-wrap: wrap-reverse;  
}
```



- Igual que `wrap`, pero invierte el orden de las filas
- En lugar de crearse filas abajo, los elementos wrapedos van arriba
- El orden visual se invierte

Ejemplo Práctico

```
<div class="contenedor-wrap">  
  <div class="item">Item 1</div>  
  <div class="item">Item 2</div>  
  <div class="item">Item 3</div>  
  <div class="item">Item 4</div>  
  <div class="item">Item 5</div>  
</div>
```



```
.contenedor-wrap {  
  display: flex;  
  flex-wrap: wrap;  
  width: 300px;  
}
```



```
.item {  
  width: 100px;
```

```
height: 50px;  
background-color: #f0f0f0;  
margin: 5px;  
}
```

Item 1

Item 2

Item 3

Item 4

Item 5

Si cambiamos la propiedad a `wrap-reverse`, el orden de las filas se invierte:

```
.contenedor-wrap {  
  display: flex;  
  flex-wrap: wrap-reverse;  
  width: 300px;  
}
```



Item 5

Item 3

Item 4

Item 1

Item 2

Y si usamos `nowrap`, todos los elementos se comprimen en una sola línea:

```
.contenedor-wrap {  
  display: flex;  
  flex-wrap: nowrap;  
  width: 300px;  
}
```



Item 1

Item 2

Item 3

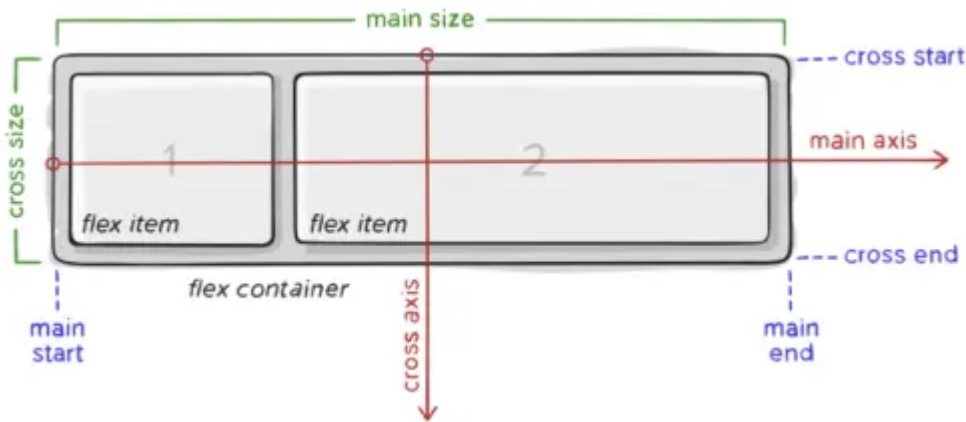
Item 4

Item 5

Dirección (flex-direction)

Flexbox trabaja con dos ejes:

- **Eje principal (main axis):** La dirección en la que se colocan los elementos flexibles. Por defecto, es horizontal (de izquierda a derecha).
- **Eje secundario (cross axis):** Perpendicular al eje principal. Por defecto, es vertical (de arriba a abajo).



La propiedad `flex-direction` establece la **dirección principal** del contenedor flexible:

```
.contenedor {
  display: flex;
  flex-direction: row; /* valor por defecto */
}
```



Valores disponibles

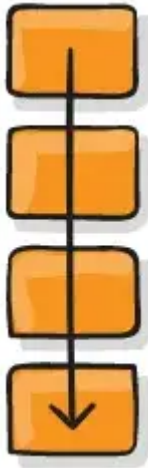
- `row`: Los elementos se colocan en fila (horizontal, de izquierda a derecha)



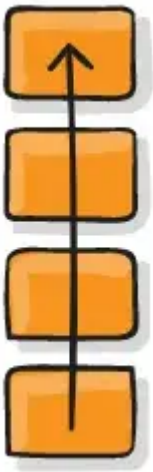
- `row-reverse`: Los elementos se colocan en fila pero en orden inverso



- `column`: Los elementos se colocan en columna (vertical, de arriba a abajo)



- `column-reverse`: Los elementos se colocan en columna pero en orden inverso



Justificación (`justify-content`)

La propiedad `justify-content` controla la alineación de los elementos a lo largo del **eje principal** del contenedor flexible (la que definimos con `flex-direction`):

```
.contenedor {  
  display: flex;  
  justify-content: flex-start; /* valor por defecto */  
}
```



Valores disponibles

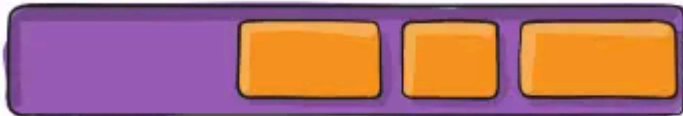
- `flex-start`: Alinea los elementos al inicio del contenedor. Es el valor por defecto.
- `flex-end`: Alinea los elementos al final del contenedor.
- `center`: Centra los elementos.
- `space-between`: Distribuye el espacio entre los elementos.

- `space-around`: Distribuye el espacio alrededor de los elementos.
- `space-evenly`: Distribuye el espacio uniformemente.

flex-start



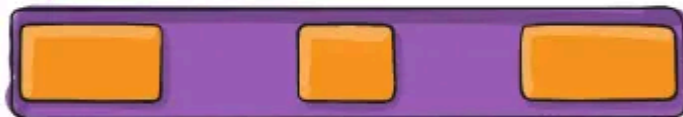
flex-end



center



space-between



space-around



space-evenly



Alineación

La alineación permite distribuir el contenido que se encuentra dentro del contenedor flexible. Para ello podemos modificar las siguientes propiedades:

- `align-content`
- `align-items`
- `align-self`

align-content

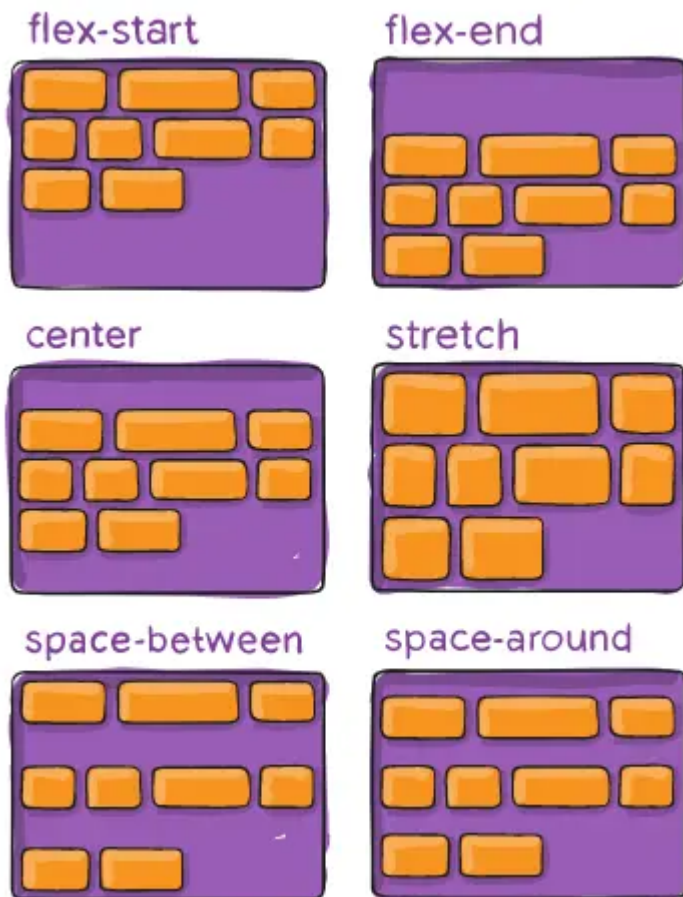
Controla la alineación de las líneas del contenedor cuando hay espacio extra en el **eje secundario**:

```
.contenedor {  
  display: flex;  
  align-content: flex-start; /* valor por defecto */  
}
```



La propiedad puede tomar los siguientes valores:

- `flex-start`: Alinea las líneas al inicio del contenedor.
- `flex-end`: Alinea las líneas al final del contenedor.
- `center`: Centra las líneas.
- `space-between`: Distribuye el espacio entre las líneas.
- `space-around`: Distribuye el espacio alrededor de las líneas.
- `space-evenly`: Distribuye el espacio uniformemente.



align-items #

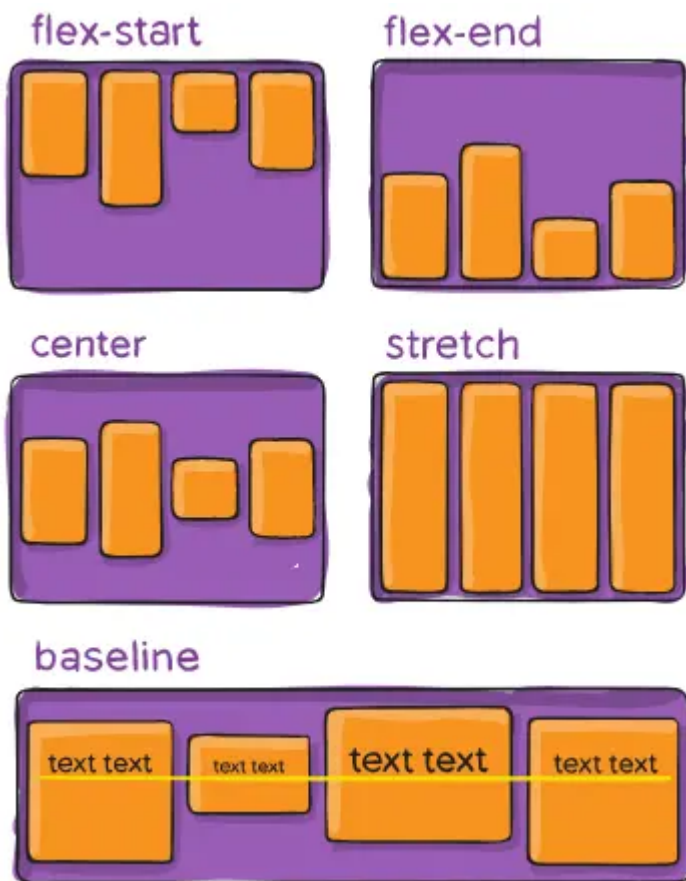
Controla la alineación de los elementos en el eje secundario. Esto es especialmente útil cuando los elementos tienen diferentes alturas:

```
.contenedor {  
  display: flex;  
  align-items: stretch; /* valor por defecto */  
}
```



Los valores posibles son:

- `stretch`: Estira los elementos para que llenen el contenedor (valor por defecto).
- `flex-start`: Alinea los elementos al inicio del contenedor.
- `flex-end`: Alinea los elementos al final del contenedor.
- `center`: Centra los elementos.
- `baseline`: Alinea los elementos según su línea base.



align-self

Permite que un elemento individual sobrescriba la alineación del contenedor:

```
.item-especial {  
  align-self: center;
```



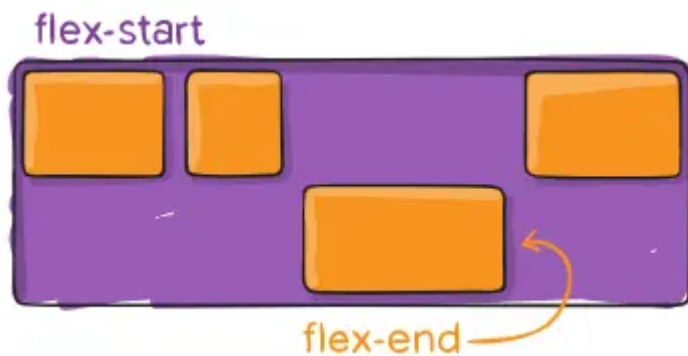
```
}
```

Los valores son los mismos que para `align-items`:

- `auto`: Hereda el valor de `align-items` del contenedor (valor por defecto).
- `stretch`: Estira el elemento para que llene el contenedor.
- `flex-start`: Alinea el elemento al inicio del contenedor.
- `flex-end`: Alinea el elemento al final del contenedor.
- `center`: Centra el elemento.
- `baseline`: Alinea el elemento según su línea base.

Por ejemplo, para alinear un ítem al final del contenedor:

```
.item-especial {  
  align-self: flex-end;  
}
```



Orden

La propiedad `order` permite cambiar el orden visual de los elementos sin modificar el HTML. El orden de los elementos se establece definiendo un valor numérico para cada uno de ellos. El valor por defecto es `0`.

```
.item {  
  order: 0; /* valor por defecto */  
}  
  
.item-primero {  
  order: -1; /* Se mostrará primero */  
}
```



```
}
```

Otros criterios de orden:

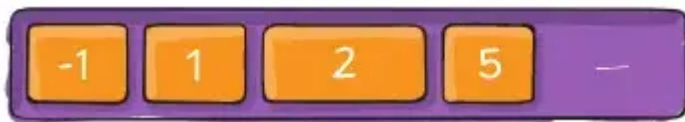
- Elementos con el mismo valor de `order` mantienen su orden original en el HTML.



- Los valores no tienen por qué ser consecutivos.



- Los valores negativos se muestran antes que los positivos.



- En el caso de trabajar con `row-reverse` o `column-reverse`, el orden visual se invierte.

Crecimiento (**flex-grow**, **flex-shrink**, **flex-basis**)

Uno de los aspectos más importantes de flexbox es entender el crecimiento/decrecimiento de un ítem. Este crecimiento va a estar condicionado por el tamaño de sus hermanos, entendiendo por hermanos aquellos ítems que están dentro del mismo contenedor. Si el ítem está solo dentro de un contenedor, no dependerá de ningún otro elemento.

flex-grow

Controla cómo crecen los elementos cuando hay espacio extra:

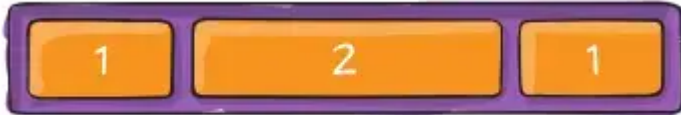
```
.item {  
  flex-grow: 0; /* valor por defecto - no crece */  
}  
  
.item-flexible {  
  flex-grow: 1; /* Puede crecer */  
}
```



Por ejemplo, si tenemos tres elementos con `flex-grow: 1`, y hay espacio extra, ese espacio se dividirá en partes iguales entre los tres elementos:



De la misma manera, si todos los elementos tienen valor 1 y uno tiene valor 2, el espacio extra se dividirá en 4 partes ($1+1+2$) y el elemento con valor 2 recibirá el doble de espacio que los otros dos:



flex-shrink

Controla cómo se encogen los elementos cuando falta espacio:

```
.item {  
  flex-shrink: 1; /* valor por defecto - puede encogerse */  
}
```



Por ejemplo, si tenemos tres elementos con `flex-shrink: 1` y el contenedor es más pequeño que la suma de los anchos de los elementos, todos se encogerán proporcionalmente para caber en el contenedor. Si, por ejemplo, hay uno con `flex-shrink: 2`, ese elemento se encogerá el doble que los otros dos.

flex-basis

Establece el tamaño inicial del elemento antes de que se distribuya el espacio libre:

```
.item {  
  flex-basis: auto; /* valor por defecto */  
}
```



Los valores pueden ser:

- `auto`: El tamaño natural del elemento.
- `0`: El elemento no tiene tamaño inicial.
- Una medida específica (px, %, em, etc.).

- `content`: El tamaño se ajusta al contenido del elemento.

Propiedad Abreviada `flex`

La propiedad `flex` es una forma abreviada de establecer `flex-grow`, `flex-shrink` y `flex-basis` en una sola línea: `flex: [flex-grow] [flex-shrink] [flex-basis];`

```
.item {  
  flex: 1 1 auto; /* flex-grow flex-shrink flex-basis */  
}
```



Esto sería equivalente a:

```
.item {  
  flex-grow: 1;  
  flex-shrink: 1;  
  flex-basis: auto;  
}
```



Herramientas para trabajar con Flexbox

Para comprender mejor cómo funcionan las diferentes opciones de configuración de Flexbox, existen varias herramientas en línea que permiten experimentar con las propiedades y ver los resultados en tiempo real. Algunas de las más populares son:

- Flexbox Froggy: Un juego interactivo que enseña los conceptos básicos de Flexbox a través de niveles desafiantes.
- CodePen: Una plataforma para escribir y compartir código HTML, CSS y JavaScript, donde puedes experimentar con Flexbox en tus propios proyectos. Concretamente puedes usar este pen para probar diferentes configuraciones de Flexbox.
- Flexbox playground: Una herramienta en línea que permite ajustar las propiedades de Flexbox y ver los resultados en tiempo real.